

Autorzy:

Gabriela Błaut
Michał Brzeziński
Aleksandra Gołek



Github projektu: <https://github.com/gabbla05/KittyProtocol>

Meowssenger / Etap 2

1. Opis funkcjonalny aplikacji

🐱 Co robi aplikacja?

Kitty Meowssenger to komunikator czasu rzeczywistego z interfejsem graficznym (GUI) stworzony w języku Go. Aplikacja realizuje model bezpiecznej, efemerycznej wymiany komunikatów tekstowych oraz statusów obecności. Wykorzystuje protokół **KittyProtocol** osadzony na warstwie transportowej **QUIC**, co gwarantuje minimalne opóźnienia, natywne szyfrowanie oraz odporność na przerwy w transmisji danych.

🐱 Jaki problem użytkownika rozwiązuje?

☆ **Prywatność i brak zaufania do infrastruktury (Zero Trust):**

Dzięki szyfrowaniu **End-to-End (E2EE)** treści wiadomości są niedostępne dla serwera pośredniczącego (Hub). Rozwiązuje to problem potencjalnego podsłuchu na poziomie administratora serwera.

☆ **Ryzyko wycieku danych historycznych:**

Komunikacja ma charakter efemeryczny (brak zapisu w bazie serwera). Wiadomości istnieją tylko w pamięci operacyjnej (RAM) aktywnych klientów, co mityguje skutki ewentualnej kompromitacji fizycznych nośników danych.

☆ **Mobilność i zmiany parametrów sieci:**

Mechanizm **Connection ID** w protokole QUIC pozwala na bezszwową migrację sesji (np. przejście z sieci Wi-Fi na 5G/LTE) bez konieczności renegotjacji połączenia (handshake), co eliminuje przestoje znane z tradycyjnych komunikatorów opartych na TCP.

Kto jest użytkownikiem (aktorzy)?

- ☆ **Użytkownik (Klient):** Inicjuje sesję, szyfruje wiadomości ustalonym wspólnym sekretami i wysyła je do Huba.
- ☆ **Centralny Hub (Serwer Sygnalizacyjny i Router):** Pełni rolę zaufanego punktu styku. Odpowiada za uwierzytelnianie (**AUTH**) oraz inteligentny routing zaszyfrowanych ramek **DATA** na podstawie identyfikatorów sesji.

2. Architektura rozwiązania

 **Model architektury:** Model **Klient-Serwer (C/S)**. Wszystkie komunikaty przechodzą przez centralny węzeł (Hub), który pełni rolę zaufanego przekaźnika (Message Router).

Komponenty systemowe:

1. **Moduł Hub (Serwer):** Zbudowany w Go, obsługuje wiele równoległych połączeń QUIC. Zintegrowany z bazą danych (np. PostgreSQL) do weryfikacji poświadczeń (AUTH). Utrzymuje w pamięci RAM mapę aktywnych sesji (login→Connection ID).

Rola: Nie deszyfruje wiadomości. Np. odbiera ramkę od Alice, sprawdza pole target, i jeśli Bob jest online, przekazuje mu paczkę.

2. **Moduł Client (Aplikacja):** Lokalny moduł kryptograficzny **na podstawie wcześniej uzgodnionego wspólnego sekretu szyfruje wiadomości** przed wysyłką (GUI).

Rola: Inicjuje połączenie do Huba, zarządza lokalnym stanem konwersacji (tylko w RAM).

Przepływ danych między komponentami:

1. Klient zestawia bezpieczny strumień QUIC/TLS 1.3 do Huba.
2. Po uwierzytelnieniu, klient prosi Hub o przesłanie statusu aktywności docelowego użytkownika (**GET_STATUS**).
3. Hub wysyła potwierdzenie o aktywności (**STATUS RES**).
4. Wiadomość jest szyfrowana lokalnie i wysyłana jako ramka **DATA** do Huba.
5. Hub odczytuje metadane (kto do kogo), ale nie treść, i przesyła ją do adresata.

3. Przypadki użycia (Use Cases)

Poniższe scenariusze udowadniają poprawność projektu i realizację wymagań protokołu.

UC1: Inicjalizacja sesji i uwierzytelnienie użytkownika

🐱 **Aktorzy:** Klient, Hub (Serwer).

🐱 **Cel:** Bezpieczne wejście do sieci i weryfikacja tożsamości.

🐱 **Przebieg techniczny:**

☆ **Handshake:**

Klient zestawia połączenie QUIC/TLS 1.3 i wysyła ramkę **HELLO** (*msg_id*). Hub odpowiada **MEOW_OK**.

☆ **Logowanie:**

Klient przesyła ramkę **AUTH** (login, hasło). Hub sprawdza hash w bazie danych.

☆ **Sukces:**

Hub odsyła **MEOW_OK** ze statusem "Logged in".

🐱 **Scenariusze błędne i reakcja UI:**

☆ **Błędne hasło (**ERR_04**):**

Hub odsyła **ERROR**. UI: Pole hasła podświetla się na czerwono, komunikat: „Nieprawidłowe dane logowania”.

☆ **Utrata połączenia w trakcie:**

Warstwa transportowa QUIC zgłasza błąd zapisu. UI: Przycisk logowania zmienia się w spinner, a po 10s pojawia się błąd: „Brak odpowiedzi serwera – sprawdź połączenie”.

☆ **Timeout (**ERR_03**):**

Brak ramki **AUTH** przez 20s. Hub zamyka sesję. UI: Powrót do ekranu startowego z informacją o wygaśnięciu sesji inicjalnej.

UC2: Bezpieczna wymiana wiadomości E2EE (End-to-End)

🐱 **Aktorzy:** Klient A (Alice), Hub, Klient B (Bob).

🐱 **Cel:** Przesłanie zaszyfrowanej treści, której serwer nie może odczytać.

🐱 **Przebieg techniczny:**

- ☆ Alice i Bob uzgadniają wspólny sekret.
- ☆ Alice za pomocą tego sekretu szyfruje ramkę **DATA** i wysyła ją do Huba. Hub potwierdza (**MEOW_OK**).
- ☆ Hub routuje ramkę do Boba. Bob deszyfruje ją przy pomocy wspólnego sekretu i odsyła aplikacyjne potwierdzenie **MEOW_OK** przez Hub do Alice.

🐱 **Scenariusze błędne i reakcja UI:**

- ☆ **Odbiorca offline (ERR_15):**
Bob wylogował się w trakcie pisania wiadomości przez Alice. Hub zwraca **ERR_15**. UI: Wiadomość Alice zostaje wyszarzona, pojawia się czerwony wykrzyknik [!] i komunikat „Odbiorca jest offline”.
- ☆ **Brak potwierdzenia (ACK failure - np. ERR_05):**
Wiadomość dotarła do urządzenia Boba, ale jego aplikacja się zawiesiła. Alice nie otrzymuje zwrotnego **MEOW_OK** w ciągu 5s. UI: Wiadomość ma status „Wysłano” (jedna fajka), ale nigdy nie zmienia się na „Dostarczono”.

UC3: Integralność danych i ochrona przed atakami

🐱 **Aktorzy:** Klient (Atakujący), Hub.

🐱 **Cel:** Ochrona Huba przed uszkodzonymi danymi i próbami przejęcia sesji.

🐱 **Przebieg techniczny:**

- ☆ **Błąd formatu (ERR_02):**
Atakujący wysyła celowo uszkodzony dokument JSON (np. brak klamry). Hub wykrywa błąd parsera i wysyła **ERR_02**.

☆ **Naruszenie stanu (ERR_01):**

Klient próbuje wysłać **DATA** przed poprawnym **AUTH**. Hub odsyła **ERR_01**.

☆ **Replay Attack (ERR_06):**

Klient wysyła ramkę z duplikatem **msg_id**. Hub odrzuca ją kodem **ERR_06**.

🐱 **Reakcja UI:**

☆ **Dla zwykłego użytkownika:**

W przypadku błędów krytycznych (np. uszkodzenie paczki przez sieć), aplikacja wyświetla: „Wystąpił błąd protokołu” i wymusza ponowne logowanie.

☆ **Dla atakującego:**

Nagłe zerwanie połączenia TLS/QUIC i zablokowanie sesji.

UC4: Kontrola ruchu i utrzymanie sesji (Traffic Control)

🐱 **Aktorzy:** Klient, Hub.

🐱 **Cel:** Zapobieganie przeciążeniu serwera i podtrzymanie łączności w trudnych warunkach sieciowych.

🐱 **Przebieg techniczny:**

☆ **Keep-alive:**

Przy braku aktywności przez 30s, Klient wysyła **PING**. Hub odpowiada **MEOW_OK**, co resetuje timery w tablicach NAT.

☆ **Rate Limiting (ERR_07):**

Użytkownik przekracza 10 wiadomości/sekundę. Hub odsyła **ERR_07** dla każdej nadmiarowej ramki.

🐱 **Reakcja UI:**

☆ **Podczas spamu:**

Pole wpisywania tekstu zostaje zablokowane (disabled) na 5 sekund. Użytkownik widzi licznik: „Piszesz zbyt szybko! Odczekaj 5s...”

☆ **W logach:**

Administrator Huba widzi flagę **RateLimit_Triggered** dla danego IP.

UC5: Obsługa limitów wielkości danych

🐱 **Aktorzy:** Klient (Nadawca), Hub.

🐱 **Cel:** Ochrona przed wysyłaniem zbyt dużych ładunków (np. próbami przesłania plików zamiast tekstu).

🐱 **Przebieg techniczny:**

- ☆ Klient przygotowuje wiadomość. Jeśli po szyfrowaniu payload przekracza 2048 bajtów, aplikacja kliencka powinna go przyciąć.
- ☆ Jeśli jednak Hub otrzyma ramkę **DATA** > 4KB (cały JSON), odrzuca ją kodem **ERR_13**.

🐱 **Reakcja UI:**

- ☆ **Prewencja:**
Licznik znaków w polu tekstowym zmienia kolor na czerwony po przekroczeniu limitu.
- ☆ **Błąd:**
Jeśli mimo to serwer odrzuci paczkę, użytkownik widzi komunikat: „Wiadomość jest zbyt długa i nie mogła zostać wysłana”.

4. Mapowanie aplikacji na protokół

Ta sekcja opisuje, w jaki sposób funkcje interfejsu graficznego (GUI) przekładają się na konkretne komunikaty wymiany danych w protokole.

🐱 **Wyszukiwanie znajomego:**

Użycie paska wyszukiwania "Search Username" powoduje wysłanie do Huba zapytania **GET_STATUS**. Jeśli użytkownik istnieje i jest online, aplikacja parsuje odpowiedź **STATUS_RES**. Na tej podstawie UI aktualizuje status użytkownika na "Online".

🐱 **Czatowanie:** Każde kliknięcie przycisku "Send" powoduje utworzenie komunikatu **DATA** zawierającego zaszyfrowany payload oraz **target**. Wiadomość jest wysyłana do Huba, który:

- ☆ odsyła **MEOW_OK** do nadawcy,
- ☆ przekazuje wiadomość do odbiorcy.

🐱 **Logowanie (Inicjalizacja):** Uruchomienie aplikacji wyzwala zestawienie połączenia QUIC i wysłanie ramki **HELLO**. Po wpisaniu danych użytkownika aplikacja wysyła **AUTH**. Pomyślne logowanie następuje po odebraniu **MEOW_OK**.

🐱 **Potwierdzenia doręczeń w UI:** Aplikacja odbiorcy po zwalidowaniu i wyświetleniu ramki **DATA** generuje **MEOW_OK** z tym samym **msg_id**, który trafia do Huba, a następnie jest przekazywany do nadawcy. Nadawca po jego otrzymaniu oznacza wiadomość jako "Delivered".

5. Wymagania pozafunkcjonalne

Aplikacja została zaprojektowana z myślą o spełnieniu kluczowych wymagań nowoczesnych usług sieciowych.

🐱 **Bezpieczeństwo:** Aplikacja wymusza użycie szyfrowania TLS 1.3 dzięki warstwie transportowej QUIC. Cała struktura JSON (w tym metadane i Payload) jest szyfrowana w locie. Mechanizm AEAD zapewnia integralność kryptograficzną, chroniąc przed atakami typu Man-in-the-Middle oraz próbami modyfikacji (Tampering).

🐱 **Wydajność:** Wykorzystanie języka Go (Golang) oraz lekkich wątków (*goroutines*) pozwala aplikacji na niemal bezkosztową obsługę połączeń. Zastosowanie QUIC eliminuje problem blokowania linii (Head-of-Line Blocking).

🐱 **Niezawodność i Odporność:** Za retransmisję zagubionych pakietów i utrzymanie ciągłości strumienia danych odpowiada natywnie biblioteka QUIC. Aplikacja implementuje aplikacyjne limity czasowe (20 sekund na uwierzytelnienie) oraz zamyka sesje po 60 sekundach bezczynności w celu ochrony zasobów.

🐱 **Skalowalność:** Hub pełni rolę routera wiadomości i nie przechowuje ich treści (E2EE), co ogranicza jego obciążenie do przekazywania zaszyfrowanych danych oraz zarządzania sesjami użytkowników.

🐱 **Logowanie i diagnostyka:** Aplikacja kliencka oraz Hub posiadają zaimplementowany mechanizm wyświetlania logów diagnostycznych w konsoli, w tym typów odebranych i wysłanych ramek JSON oraz zdarzeń sieciowych.

6. Plan implementacji i testowania

Zgodnie z wymaganiami, aplikacja jest rozwijana iteracyjnie, z jasnym podziałem na role i testy .

Zakres MVP (Minimum Viable Product):

- ☆ Działający Serwer (Hub) przyjmujący połączenia QUIC, weryfikujący loginy i routujący wiadomości.
- ☆ Klient w formie aplikacji okienkowej (GUI - Kitty Meowssenger) potrafiący połączyć się z Hubem, a następnie komunikować się z innymi użytkownikami za pośrednictwem Huba przy użyciu komunikatów **DATA**.
- ☆ Wysyłanie i odbieranie tekstowych wiadomości **DATA** wraz ze zautomatyzowanym odsyłaniem potwierdzeń **MEOW_OK**.
- ☆ Rygorystyczny parser JSON odrzucający uszkodzone struktury błędem **ERR_02**.

Plan testów:

Testy będą przeprowadzane manualnie w izolowanym środowisku lokalnym (localhost) przy użyciu wielu instancji terminala (w celu symulacji oddzielnych węzłów i portów sieciowych) .

- ☆ **Testy funkcjonalne:**
Weryfikacja ścieżki "Happy Path" - logowanie dwóch hostów i udana wymiana wiadomości.
- ☆ **Testy błędów:**
Próba komunikacji z nieistniejącym użytkownikiem (weryfikacja **ERR_16**) oraz wpisanie błędnego hasła.
- ☆ **Testy bezpieczeństwa:**
Podłączenie się do aplikacji i próba wstrzyknięcia uszkodzonego ciągu bajtów zamiast struktury JSON (weryfikacja poprawnego odrzucenia ramki **ERR_02**) .
- ☆ **Testy jakościowe:**
Sprawdzić jak jakość maszyny, na której stoi serwer (ilość rdzeni) wpływa na jakość routowania i ogólnego funkcjonowania serwisu



Podział pracy w zespole:

☆ **Osoba 1 (Backend & Hub):**

Implementacja Serwera Sygnalizacyjnego w języku Go, zarządzanie mapą poświadczeń i routowaniem adresów IP w modelu współbieżnym.

☆ **Osoba 2 (Client & UI):**

Zaimplementowanie interfejsu graficznego (front-end okna czatu, pasek boczny), obsługa zdarzeń użytkownika (kliknięcia, wpisywanie tekstu), logika nawiązywania bezpośredniego tunelu QUIC i aktualizacja widoku rozmowy.

☆ **Osoba 3 (Dokumentacja & QA & testowanie):**

Opracowanie pełnej dokumentacji technicznej, walidatorów parsera JSON, wygenerowanie i przypisanie testowych certyfikatów TLS 1.3 oraz przygotowanie prezentacji.